SOLID Principles Analysis - Vehicle Management System

SOLID Principles Compliance Report

S - Single Responsibility Principle

Status: GOOD #

Each class has a single, well-defined responsibility:

Model Layer:
- Vehicle class: Data representation only
- User, Driver, Manager: Entity data
- Trip, Maintenance: Transaction data

DAO Layer:
- VehicleDaoImpl: Database operations for Vehicle only
- DriverDaoImpl: Database operations for Driver only
- Each DAO has one reason to change: persistence logic

Service Layer:
- VehicleService: Business logic for vehicles
- DriverService: Business logic for drivers
- AuthService: Authentication business logic only

Controller Layer:
- VehicleController: Handles vehicle UI logic
- DriverController: Handles driver UI logic
- AuthController: Handles authentication UI logic

Presentation Layer:
- VehiclesPanel: Vehicle table UI only
- DriversPanel: Driver table UI only
- LoginFrame: Login UI only

 # Benefits Achieved:
- Easy to maintain and update
- Clear responsibility boundaries
- Reduced code coupling

O - Open/Closed Principle

Status: GOOD #

Classes are open for extension, closed for modification:

Interface-based Design:
- VehicleService interface allows new implementations
- DriverService interface extensible for new features
- VehicleDao interface allows different DB implementations

Inheritance Hierarchy:
- Vehicle (abstract) ← Bus, Car, Truck
  Can extend with new vehicle types without modifying existing code
- User (abstract) ← Driver, Manager
  Can extend with new user roles without changes

Implementation Separation:
- Services are interfaces
- DAOs are interfaces
- Allows swapping implementations without affecting clients

Example:
If you want to add caching layer:
- Create CachedVehicleDaoImpl implements VehicleDao
- No changes needed to existing code

 # Benefits Achieved:
- New features via extension, not modification
- Reduced risk of breaking existing functionality
- Easy to scale new vehicle/user types

L - Liskov Substitution Principle

Status: GOOD #

Derived classes can substitute their base classes:

Vehicle Hierarchy:
- Bus, Car, Truck can replace Vehicle
- All inherit from Vehicle properly
- Example: List<Vehicle> can contain Bus/Car/Truck and work correctly

User Hierarchy:
- Driver, Manager properly extend User
- Can be used wherever User is expected
- getRole(), getName() work correctly for all subtypes

Service Implementations:
- VehicleServiceImpl properly implements VehicleService
- Can be used wherever VehicleService is expected
- All interface contracts honored

DAO Implementations:
- VehicleDaoImpl properly implements VehicleDao
- All CRUD operations work as expected

 # Benefits Achieved:
- Polymorphism works correctly
- Flexible code that works with base types
- Inheritance used properly, not violated

Project: Vehicle Management System (VMS)
Analysis Date: February 20, 2026
Status: Partially Compliant with Areas for Improvement

I - Interface Segregation Principle

Status: GOOD #

Clients depend only on interfaces they use:

Segregated Service Interfaces:
- VehicleService: Only vehicle operations
  (saveVehicle, getVehicleById, getAllVehicles, updateVehicle, deleteVehicle)
- DriverService: Only driver operations
  (saveDriver, getDriverById, getAllDrivers, getAvailableDrivers, updateDriver, deleteDriver)
- AuthService: Only auth operations
  (login, logout)

Segregated DAO Interfaces:
- Each DAO interface is specific to one entity
- Controllers only depend on interfaces they need

Advantages:
- VehicleController doesn't know about DriverService
- AuthController doesn't depend on VehicleService
- Clean dependency graph
- No forced dependencies on unused interfaces

Current Interface Design:
Small, focused interfaces
No "fat" interfaces
Each interface has clear purpose

 # Benefits Achieved:
- Reduced coupling between components
- Easier to test in isolation
- Clear contracts

D - Dependency Inversion Principle

Status: NEEDS IMPROVEMENT ##

Issue: Controllers and Services create dependencies directly

PROBLEM #1: Service Creation in Controller

Current Code (VehicleController):
```
public VehicleController(VehiclesPanel view) {
    this.view = view;
    this.vehicleService = new VehicleServiceImpl(); // # VIOLATION
}
```

Problem:
- Controller depends on concrete VehicleServiceImpl
- Violates DIP (should depend on abstraction)
- Hard to test (can't inject mock service)
- Tight coupling to implementation

PROBLEM #2: DAO Creation in Service

Current Code (VehicleServiceImpl):
```
public VehicleServiceImpl() {
    this.vehicleDao = new VehicleDaoImpl(); // # VIOLATION
}
```

Problem:
- Service depends on concrete VehicleDaoImpl
- Hard to swap implementations
- Violates DIP

IMPACT:
Hard to unit test
Hard to mock services/DAOs
Tight coupling throughout application
Difficult to change implementations

 # Current Positive Aspects:
Using interfaces as variable types
Dependency references as finals (immutable)
Clear separation exists

RECOMMENDATIONS & IMPROVEMENTS

Fix #1: Constructor Injection in Controllers

Change from:
```
public VehicleController(VehiclesPanel view) {
    this.vehicleService = new VehicleServiceImpl();
}
```

Change to:
```
public VehicleController(
    VehiclesPanel view,
    VehicleService vehicleService) {
    this.view = view;
    this.vehicleService = vehicleService;
}
```

Benefits:
Easier dependency injection
Can inject mocks for testing
Follows DIP
More flexible

Fix #2: Constructor Injection in Services

Change from:
```
public VehicleServiceImpl() {
    this.vehicleDao = new VehicleDaoImpl();
}
```

Change to:
```
public VehicleServiceImpl(VehicleDao vehicleDao) {
    this.vehicleDao = vehicleDao;
}
```

Benefits:
Can inject different DAO implementations
Mockable for testing
Follows DIP
Allows caching layer in future

Fix #3: Factory Pattern or Dependency Injection Container

Option A: Factory Pattern

```
public class ServiceFactory {
    public static VehicleService createVehicleService() {
        VehicleDao dao = new VehicleDaoImpl();
        return new VehicleServiceImpl(dao);
    }
}

// Usage:
VehicleService service = ServiceFactory.createVehicleService();
```

Option B: Spring Dependency Injection (Recommended)

```
@Service
public class VehicleServiceImpl implements VehicleService {
    @Autowired
    private VehicleDao vehicleDao;
}

@RestController
public class VehicleController {
    @Autowired
    private VehicleService vehicleService;
}
```

Option C: Manual Dependency Injection

```
// In main application initialization:
VehicleDao vehicleDao = new VehicleDaoImpl();
VehicleService vehicleService = new VehicleServiceImpl(vehicleDao);
VehicleController controller =
    new VehicleController(vehiclesPanel, vehicleService);
```

OVERALL ASSESSMENT

Current State: 8/10 - Good with Room for Improvement

Strengths:
Excellent layer separation
Good use of interfaces and abstractions
Strong adherence to SRP, OCP, LSP, ISP
Clear hierarchy and inheritance patterns
Service/DAO separation is clean

Weaknesses:
Dependency Inversion needs fixing
Direct instantiation of implementations
Limited testability due to tight coupling
No dependency injection framework

Priority Fixes (in order):
1. **HIGH:** Add constructor injection for services in controllers
2. **HIGH:** Add constructor injection for DAOs in services
3. **MEDIUM:** Implement Factory Pattern or Spring DI
4. **MEDIUM:** Add dependency injection container
5. **LOW:** Add validation layer

Overall Verdict:
The project has a SOLID foundation with clean architecture. With the recommended improvements to dependency inversion, the code will be more testable, maintainable, and scalable.

Estimated Effort to Full Compliance: 2-3 hours